

CSCI 2503 — Assignment 5

C++ Structs & Learning Methodology

Brian Abbott · Survey of Programming Languages · Due June 25, 2026

Problem 1 — C++ Structs

The working C++ struct, the test program, and a screenshot of the program's output are included in the accompanying zip file. I wrote my struct to make its data members private (which overrides the struct's default public visibility), and I added getters and setters, so the struct faithfully replicated the Java class's encapsulation. Please see the answers below.

Comparing the C++ struct to the C# class

Rewriting the Fan class as a C++ struct surprised me with how little I had to change. YouTuber The Cherno has a YouTube video that was a good starting point (<https://www.youtube.com/watch?v=fLgTtaggJp0>). The only difference, according to him, was that by default, properties are private in a class, but public in a struct. However, you can specify a visibility modifier. Furthermore, he asserted convincingly that structs were only added to C++ to maintain C backward compatibility with C, and that choosing whether to use a class/struct was somewhat arbitrary, and based on your coding style.

The bottom line is that everything I thought a struct didn't have – constructors, member functions, access specifiers, static members, inheritance, operator overloads, virtual functions - is available to both structs and classes. Recreating the Fan as a struct thus meant taking the same members and methods and simply changing the default public visibility to private.

What was easier in C# (and a bit more work in C++):

- C# properties (`public double Radius { get; set; }`) collapse the getter/setter pair into a single declaration with an auto-generated backing field. C++ has no property syntax, so I wrote each accessor and mutator out by hand - more boilerplate for the same result.
- C# standardizes `ToString()` as an override that every type inherits from `System.Object`. C++ has no universal base class, so a "to-string" is only a convention: I named a `toString()` member (overloading `operator<<` is the other idiom), and nothing in the language enforces or recognizes it.
- C# records (from Assignment 4) give you value equality and concise immutable types for free. C++ has no record type; you approximate it with a defaulted `operator==` (C++20) and `const` members.

What was cleaner in C++:

- For a plain data holder, the struct needs nothing at all - three members with default member initializers and you're done, with no ceremony. Adding a default property is also pretty simple.

- Default member initializers (`int speed{1};`, etc., LearnCPP 13.9) supply the no-arg defaults declaratively, rather than in a constructor body, the way the Java no-arg constructor did.

The biggest contrast, though, is conceptual rather than syntactic: **the same keyword means very different things in the two languages**. In C#, `struct` is a value type (copied by value, lives on the stack, cannot be a base class), while `class` is a reference type (heap-allocated, garbage-collected, compared by reference). Choosing one over the other in C# changes the semantics of copying, identity, and nullability. In C++, choosing `struct` over `class` changes essentially nothing except who may touch the members by default - both are value types, both can live on the stack or the heap, both can be inherited from. So “struct vs. class” is a major design decision in C# and an almost cosmetic one in C++, resulting from a design choice to allow business owners to adopt C++ without losing their existing C code. (video on why C++ beat Pascal at LearnCPP 13.9).

Do you lose any wanted functionality with the non-class approach?

No. Because a C++ `struct` can do everything a C++ `class` can, there is no capability I had to give up. Encapsulation, validation inside setters, inheritance, polymorphism, operator overloading - all of it is still available in a `struct`; I just write it. The only thing I “lose” by choosing `struct` is the default: members start public instead of private. That default is one line away from being changed, and in this assignment, I did change it - a single “private:” line above the data members gives the `struct` the same encapsulation the Java `class` had.

Encapsulation, and what a struct’s field exposes, that a class can prevent

Encapsulation still works. The difference is the default: a `struct` **opts out** of encapsulation, while a `class` **opts in**.

That default is precisely the thing a `class` can prevent. Left at the `struct`’s public default, the fields would let outside code reach in and write `fan.speed = -100;` directly, with no validation. This is exactly why I marked them private in my submission. With the members private, callers must go through `setSpeed`, where I could clamp or reject bad values. A related capability is **aggregate initialization**: a plain `struct` (all-public members, no user-declared constructors, no base classes, no virtual functions) is an aggregate, so `Fan f{3, 8.0, "blue"};` works with no constructor written. A `class` with private members is not an aggregate, so that brace form is prevented. The same is true of my `struct`, since its members are private and it declares its own constructors.

One nuance worth noting: the moment I add my own constructor - such as the convenience constructor that takes a radius - the `struct` stops being an aggregate too, so aggregate initialization and explicit constructors are somewhat mutually exclusive. That trade-off is why I had to keep `Fan() = default;`, which preserves default-initialization through the member initializers while still allowing the radius constructor.

Is there any part of the class you could not replicate?

No. Every part of the class-based Fan can be replicated in a struct, because in C++ the two are the same feature with different defaults. I could literally take a `class Fan { ... };`, rename the keyword to `struct`, add `public:` / `private:` to match the original access, and the program would behave identically.

Problem 2 — Learning Methodology

In my experience, yes — coming from Java, getting started in C#, C++, and C was noticeably faster than starting a language with a different shape like Python.

The C-family languages share so much surface area with Java that a great deal was transferred for free: curly-brace blocks, semicolons, the same `if/for/while` syntax, static typing, and the class and OOP vocabulary I already had. C# was the smoothest of all in some ways - close enough to Java that I was productive almost immediately, and most of what I had to learn were not genuinely new concepts. Obviously, C# syntax carried over neatly, along with the concepts it borrowed from Java. But I'm more comfortable with Unix environments, and while namespaces were similar enough to packages (and I've written languages that had namespaces before) assemblies confused me. I don't spend much time in the Windows world, and I didn't have a grasp of concepts like DLLs, etc.

C++ shared the syntax and the OOP model, but layered on things Java had deliberately hidden from me - pointers, references, manual memory and object lifetime, the header-versus-implementation split, the build model - so the start was fast, but the depth took real work. Furthermore, C++ had a different syntax compared to Java.

C, by way of K&R, stripped the OOP scaffolding away entirely and made me reason about memory and pointers directly, which was the opposite of “easy,” but it built understanding I am starting to lean on. Right now, I am generally reading textbooks recommended by this course. I do feel strongly about typing every line of code that a teacher gives, if at all possible. So I plan to work on these languages some more, starting with K&R's C, and moving towards C++, both from <https://learncpp.com>, as well as Tour of C++, which covers C++20, which I borrowed the `std::format` instead of `printf`.

Python is an interesting case because it is genuinely easy to start regardless of background - the syntax is minimal and readable - but its idioms (dynamic typing, duck typing, comprehensions, and an overall “Pythonic” style) are different enough that my Java habits sometimes worked against me. The on-ramp is gentle for everyone, so my Java background actually helped me less there than it did with C#. I also cursed under my breath when I read how Python handled “truthiness”.

So my honest take is that syntactic and OOP familiarity from Java transferred strongly to C#, decently to C++, and least to languages defined by a different paradigm. The start was quicker for the C-family because I was reusing patterns I already knew; the real learning curve depended far less on the syntax and far more on what new ideas each language forced me to absorb: manual memory in C and C++, and a simple, idiomatic style in Python that looks closer

to what someone who writes a lot of Python would write. Finally, and I'm not just saying this to be cool, because I think it's cliché, C and C++ are coming easier than C#. Similar syntax is one thing, but I would have to spend more time in the Windows world to better understand C#. I'm planning to write a C# program to pull data from a VTScada server one day to better familiarize myself with the Windows world. If you're too cool for Windows, you're too cool for a lot of work.